

```

var currentBaseType = 0;
var ext_m7 = false;
var ext_M7 = false;
var ext_6 = false;
var ext_9 = false;

// Keeps a strict chronological log of the exact pitches generated per performance key
var activePitchesMemory = {};

var PluginParameters = [
  { name: "Control Octave", type: "menu", valueStrings: ["C0 (MIDI 24)", "C1 (MIDI 36)", "C2 (MIDI 48)", "C3 (MIDI 60)"], defaultValue: 2 },
  { name: "Triad Voicing", type: "menu", valueStrings: ["Root Position", "1st Inversion", "2nd Inversion"], defaultValue: 0 },
  { name: "Auto-Bass Register", type: "menu", valueStrings: ["OFF", "C0 Range (Sub)", "C1 Range (Low)", "C2 Range (Mid)"], defaultValue: 2 },
  { name: "Strum Engine", type: "menu", valueStrings: ["OFF", "ON"], defaultValue: 1 },
  { name: "Strum Direction", type: "menu", valueStrings: ["Low to High (Up)", "High to Low (Down)"], defaultValue: 0 },
  { name: "Strum Speed (ms)", type: "linear", minValue: 0, maxValue: 360, numberOfSteps: 360, defaultValue: 40 },
  { name: "Strum Size", type: "menu", valueStrings: ["1 Octave", "2 Octaves", "3 Octaves"], defaultValue: 0 }
];

function HandleMIDI(event) {
  // --- DYNAMIC CONTROL ZONE ENGINE ---
  var controlOctaveSetting = GetParameter("Control Octave");
  var controlLowBound = 24 + (controlOctaveSetting * 12);
  var controlHighBound = controlLowBound + 11;

  // --- 1. THE DYNAMIC CONTROL ZONE WALL ---
  if (event.pitch >= controlLowBound && event.pitch <= controlHighBound) {
    if (event instanceof NoteOn) {
      var noteOffset = event.pitch - controlLowBound;

      if (noteOffset === 0) { currentBaseType = 0; clearExtensions(); } // Root Key (C)
      if (noteOffset === 2) { currentBaseType = 1; clearExtensions(); } // Minor Key (D)
      if (noteOffset === 4) { currentBaseType = 2; clearExtensions(); } // Sus4 Key (E)
      if (noteOffset === 5) { currentBaseType = 3; clearExtensions(); } // Dim Key (F)

      if (noteOffset === 1) { ext_m7 = !ext_m7; if(ext_m7) ext_M7=false; } // C#
      if (noteOffset === 3) { ext_M7 = !ext_M7; if(ext_M7) ext_m7=false; } // D#
      if (noteOffset === 6) { ext_6 = !ext_6; } // F#
    }
  }
}

```

```

        if (noteOffset === 8) { ext_9 = !ext_9; } // G#

        if (noteOffset === 11) { clearExtensions(); } // B
    }
    return;
}

// --- 2. PERFORMANCE ZONE ---
var voicingSetting = GetParameter("Triad Voicing");
var bassSetting = GetParameter("Auto-Bass Register");
var strumOn = GetParameter("Strum Engine");
var strumDirection = GetParameter("Strum Direction");
var strumDelayMs = GetParameter("Strum Speed (ms)");
var octaveSizeSetting = GetParameter("Strum Size");

// --- EXECUTE PERFORMANCE NOTE ON ---
if (event instanceof NoteOn) {
    // Core foundation triad tracking arrays
    var coreTriad = [0];

    if (currentBaseType === 0) { coreTriad.push(4, 7); } // Major (1, 3, 5)
    if (currentBaseType === 1) { coreTriad.push(3, 7); } // Minor (1, b3, 5)
    if (currentBaseType === 2) { coreTriad.push(5, 7); } // Sus4 (1, 4, 5)
    if (currentBaseType === 3) { coreTriad.push(3, 6); } // Dim (1, b3, b5)

    // Apply Upward Triad Inversion Math
    if (voicingSetting === 1) {
        coreTriad[0] += 12;
    } else if (voicingSetting === 2) {
        coreTriad[0] += 12;
        coreTriad[1] += 12;
    }
}

// Initialize final interval pool with the inverted triad notes
var baseIntervals = [];
for (var t = 0; t < coreTriad.length; t++) {
    baseIntervals.push(coreTriad[t]);
}

// Layer extensions strictly over top
if (ext_m7) { baseIntervals.push(10); }
if (ext_M7) { baseIntervals.push(11); }
if (ext_6) { baseIntervals.push(9); }
if (ext_9) { baseIntervals.push(14); }

```

```

// Multi-octave replication logic
var finalIntervals = [];
var totalOctaves = octaveSizeSetting + 1;

for (var oct = 0; oct < totalOctaves; oct++) {
  for (var i = 0; i < baseIntervals.length; i++) {
    var calculatedInterval = baseIntervals[i] + (oct * 12);
    if (finalIntervals.indexOf(calculatedInterval) === -1) {
      finalIntervals.push(calculatedInterval);
    }
  }
}

// Dynamic Directional Array Sorting
if (strumDirection === 1) {
  // High to Low (Descending Sort)
  finalIntervals.sort(function(a, b){return b - a});
} else {
  // Low to High (Ascending Sort)
  finalIntervals.sort(function(a, b){return a - b});
}

// Dynamic Bass Assignment
var bassPitch = event.pitch;
if (bassSetting === 1) {
  while (bassPitch >= 24) { bassPitch -= 12; }
} else if (bassSetting === 2) {
  while (bassPitch >= 36) { bassPitch -= 12; }
  while (bassPitch < 24) { bassPitch += 12; }
} else if (bassSetting === 3) {
  while (bassPitch >= 48) { bassPitch -= 12; }
  while (bassPitch < 36) { bassPitch += 12; }
}

activePitchesMemory[event.pitch] = [];

// Broadcast Auto-Bass Voice (Always fires instantly as the foundational anchor)
if (bassSetting > 0) {
  var bassNote = new NoteOn(event);
  bassNote.pitch = bassPitch;
  bassNote.send();
  activePitchesMemory[event.pitch].push({ pitch: bassPitch, delayIndex: 0 });
}

```

```

// Broadcast Strummed Chord Array
for (var j = 0; j < finalIntervals.length; j++) {
    var targetPitch = event.pitch + finalIntervals[j];
    activePitchesMemory[event.pitch].push({ pitch: targetPitch, delayIndex: j });

    var outNote = new NoteOn(event);
    outNote.pitch = targetPitch;

    if (strumOn === 1) {
        outNote.sendAfterMilliseconds(j * strumDelayMs);
    } else {
        outNote.send();
    }
}
}

// --- EXECUTE PERFORMANCE NOTE OFF ---
else if (event instanceof NoteOff) {
    if (activePitchesMemory[event.pitch]) {
        var notesToKill = activePitchesMemory[event.pitch];

        for (var k = 0; k < notesToKill.length; k++) {
            var outNoteOff = new NoteOff();
            outNoteOff.pitch = notesToKill[k].pitch;
            outNoteOff.velocity = 0;

            if (strumOn === 1 && notesToKill[k].delayIndex > 0) {
                outNoteOff.sendAfterMilliseconds(notesToKill[k].delayIndex * strumDelayMs);
            } else {
                outNoteOff.send();
            }
        }
        delete activePitchesMemory[event.pitch];
    }
}

function clearExtensions() {
    ext_m7 = false; ext_M7 = false; ext_6 = false; ext_9 = false;
}

```